

MLOPS PORTFOLIO PROJECT · CORRECTED EDITION

Automated Support Ticket Classification

The 30-Day Build Guide, with every known defect fixed

Supersedes both earlier PDFs

Automated-Support-Ticket-Classification-MLOps-30-Day-Guide.pdf

Ticket-Triage-MLOps-30-Day-Guide.pdf

Windows / PowerShell · one commit per day

Package: `automated_support_ticket_classification`

All 30 days complete · tagged v1.0.0

Plus three post-release features, see Part 8

49 commits · runs locally by design

Revised 25 September 2026

Contents

- Part 0 **Read this first.** Why this edition exists, which of the two source guides wins, and where the project finished.
- Part 1 **Errata.** Thirty-three verified defects across the two guides, the repository and the data. Thirty are resolved and marked as such.
- Part 2 **Repair sprint.** The nine-step sequence that brought the repo to a clean baseline. All complete; kept for rebuilding from scratch.
- Part 3 **Days 14 to 30, corrected.** Every day with the fixes folded in and a note on what actually happened when it was built.
- Part 4 **Reference files.** Final, correct contents of every config file in one place.
- Part 5 **Troubleshooting.** Expanded Windows table, including every failure actually hit.
- Part 6 **Verification matrix.** What to run and what to expect, per day, plus the final checklist.
- Part 7 **Interview preparation.** The core questions, plus the stronger answers this build earned.
- Part 8 **Beyond Day 30.** A real corpus, a model registry with gated promotion, and drift monitoring. Each shipped with a defect, and each defect is the more interesting half.
-

How this edition differs

Every claim in this document was reproduced on the machine before it was written down. Where a fix involves a version pin, the boundary version was checked against the actual package source rather than assumed. Nothing here is a guess, and where something is cosmetic rather than broken, it says so. **There are no unverified claims left.**

Outcome. Thirty days built, tagged `v1.0.0` and released, then extended. 49 commits, 30 tests, CI green on every run. Trained on 13,083 real support messages: accuracy 0.9282, macro F1 0.9171 across five classes.

This project runs locally by design. It is a portfolio build, not a hosted service, so it was deliberately never deployed. The deployment machinery from Days 28 and 29 is present, working and inactive. That is a scope decision, not an unfinished task, and this edition describes it that way throughout.

Part 0 · Read this first

You were given the same guide twice

The two source PDFs are not two guides. They are one 82-page guide published under two package names. A normalised line-by-line comparison of the extracted text found **zero** substantive differences: same 30 days, same code, same commands, same interview questions. The only variations:

Item	Automated-Support-Ticket... PDF	Ticket-Triage PDF
Python package	<code>automated_support_ticket_classification</code>	<code>ticket_triage</code>
Repo / image name	<code>automated-support-ticket-classification</code>	<code>ticket-triage</code>
GitHub handle	hard-coded as <code>raviteja311</code>	left as <code><user></code>
Render URL	<code><YOUR_RENDER_URL></code>	<code>https://ticket-triage.onrender.com</code>

Canonical naming. The repo is `Automated-Support-Ticket-Classification` and the package is `automated_support_ticket_classification`. Archive the Ticket-Triage PDF so you never half-follow it and hit `ModuleNotFoundError: No module named 'ticket_triage'`.

Where the project finished

Signal	State	Detail
Progress	30 of 30	Tagged and released as <code>v1.0.0</code> , then extended. 49 commits.
Test suite	30 passed	Data, corpus mapping, model, promotion gate, drift, API and UI.
Lint and format	clean	<code>ruff check .</code> and <code>ruff format --check .</code> both pass.
CI history	100% green	Every run succeeded: format, lint, tests, image build, container smoke test.
Pipeline	locked	Four DVC stages, <code>dvc.lock</code> committed, artifacts pushed to the remote.
Model	0.9282 / 0.9171	Accuracy and macro F1 on real support text (banking77). See Part 8.
Service	verified locally	Run with <code>uvicorn</code> ; all five classes classified correctly, <code>/metrics</code> accumulating.
Container	builds	711 MB image, 3.26 kB build context, <code>\$PORT</code> honoured.
Deployment	by design	Deliberately not deployed. Blueprint and smoke workflow present but inactive.
Prometheus scrape	verified	Target UP, scraping <code>http://api:8000/metrics</code> , counters stored and rising.
Beyond Day 30	3 features	Real corpus, model registry with gated promotion, drift monitoring. Part 8.

Verified by running it, not only by tests

The service was started with `uvicorn` and exercised end to end. One real ticket per class, all five correct:

Ticket	Predicted	Confidence
--------	-----------	------------

I need a refund for the duplicate payment of \$49	billing	0.938
The app crashes every time I open the reports page	technical	0.902
I cannot reset my password, the email never arrives	account	0.839
My order #10231 has not arrived after two weeks	shipping	0.893
What are your customer support working hours	general	0.866

`/metrics` accumulated real counters across that traffic, so the Prometheus instrumentation itself is confirmed working.

The monitoring stack, verified end to end

Docker Desktop initially refused to start on this machine: its `docker-desktop` WSL distro would boot and immediately stop. A full restart cleared it, and the stack was then run and observed:

```
job:      automated-support-ticket-classification
scrape_url: http://api:8000/metrics
health:    UP
last error: (none)
```

Six requests were sent to the API, then Prometheus was queried for what it had *stored*, rather than trusting the target status alone:

```
/health  GET  2xx  value=3
/metrics  GET  2xx  value=2    <- Prometheus scraping the API
/predict  POST  2xx  value=5    <- exactly the 5 predictions sent
```

The counts match the traffic sent exactly. Waiting a further scrape interval showed the `/metrics` counter rise from 2 to 5, which proves the loop is live rather than a single lucky connection. All three RED signals are present: 3 counter series for rate and errors, and 12 histogram buckets for duration.

Every claim in this document has now been executed and observed. Nothing is asserted on trust.

On deployment

Days 28 and 29 produced a working Render blueprint and a post-deploy smoke workflow. Neither is active, because this project runs locally by choice. The smoke workflow is manual-only: its daily schedule is commented out, since on a timer it would fail every morning on a missing `SERVICE_URL` and leave the Actions tab permanently red, which reads as a broken project rather than an undeployed one. To deploy later, uncomment the schedule and set the secret.

Severity key

CRITICAL stops progress. **HIGH** silently produces a wrong or unprofessional result. **MEDIUM** real but survivable. **COSMETIC** the guide says something untrue but nothing breaks. **FIXED** resolved in the repository.

Part 1 · Errata

Thirty-three defects. Thirty are fixed and marked **FIXED**; the diagnosis is kept because the reasoning is the valuable part and most of it is interview material. Group A is the repository as found, Group B the guides themselves (each tied to the day it detonates), Group C documentation that is simply wrong but harmless, and Group D defects discovered in the data and code while building.

Summary

ID	Status	Where	One-line statement
A1	FIXED	tests/	A test imported <code>models/train.py</code> , a file that never existed.
A2	FIXED	models/	<code>models/__init__.py</code> made an artifact directory importable.
A3	FIXED	pyproject.toml	<code>description</code> sat in <code>[build-system]</code> instead of <code>[project]</code> .
A4	FIXED	tests/	Lint and format were failing, which would have reddened Day 25 CI.
A5	FIXED	data/	Three CSVs were tracked by Git, which DVC refuses to take over.
A6	FIXED	.gitignore	<code>metrics/metrics.json</code> was ignored, defeating Day 12 and Day 16.
A7	FIXED	environment	DVC 3.55.2 was incompatible with the installed pathspec 1.1.1.
A8	FIXED	environment	scikit-learn and scipy disagreed, warning on every model fit.
A9	FIXED	environment	Pydantic deprecation warnings from MLflow drowned the test output.
A10	FIXED	environment	Python 3.12 locally, 3.11 in Docker and CI.
B1	FIXED	Day 2	The day ignores <code>data/</code> then runs <code>git add data</code> . It exits 1.
B2	FIXED	Day 3	<code>pip install -e .</code> runs before any <code>__init__.py</code> exists.
B3	FIXED	Day 3	PDF reflow puts TOML keys in the wrong tables.
B4	FIXED	Day 3	Pins 11 packages but not the 3 transitive ones that actually break.
B5	FIXED	Day 6	<code>Path("params.yaml")</code> is relative to the working directory.
B6	FIXED	Day 6	Pydantic v2 warns on every <code>model_</code> field name.
B7	FIXED	Day 10/11	Metrics stay numpy scalars, which are not JSON serialisable.
B8	FIXED	Day 11	<code>mlflow</code> may be <code>None</code> where static analysis can see it.
B9	FIXED	Day 16	DVC stages omit <code>config.py</code> , so config edits trigger no rerun.
B10	FIXED	Day 20	The API cleaned text differently from training. Train/serve skew.
B11	FIXED	Day 20/22	The empty-text branch is unreachable; whitespace-only was untested.
B12	FIXED	Day 23	The lean image installed MLflow and DVC, contradicting Day 11.
B13	FIXED	Day 25	CI assumes the venv is <code>.venv</code> ; this one is <code>ML0ps</code> .
B14	FIXED	Day 15	A Windows backslash path gets committed into <code>.dvc/config</code> .
B15	FIXED	Day 26/30	Badge URLs use a repo name that does not match the real one.
B16	FIXED	Day 20	<code>zip()</code> without <code>strict=</code> violates a rule Day 4 enables.
B17	FIXED	Day 29	The smoke test cannot survive a free-tier cold start.

B18	FIXED	Day 3	Declares <code>pydantic-settings</code> , which nothing in the project imports.
C1	COSMETIC	Day 4	Claims files were reformatted when the repo has no Python files yet.
C2	COSMETIC	Day 2	Claims 5 files changed, unreachable given B1.
C3	COSMETIC	Day 2	<code>.github/workflows/</code> is created but never committed.
C4	FIXED	whole guide	Two PDFs, two package names, one project.
D1	FIXED	data	The dataset had four classes, not five: <code>shipping</code> missing, <code>account</code> contaminated.

C1, C2 and C3 remain open because they are statements in the source PDFs that nothing in a repository can fix. They are listed so the expected output does not confuse you.

Group A · Defects found in the repository

All ten are fixed. The diagnosis is kept because understanding why each happened is worth more than the one-line fix, and several are strong interview material.

A1 **FIXED** A test pointed at files that never existed

SYMPTOM. `FileNotFoundError: ...\models\train.py` from `tests/test_model_scripts.py`.

ROOT CAUSE. The test loaded `models/train.py` and `models/evaluate.py` from the repo root by file path. Neither exists and neither should: the code lives in `src/automated_support_ticket_classification/models/`, while root-level `models/` is the DVC output directory holding `model.joblib`. Notably, **this test appears in neither PDF**; both suites are exactly `test_data.py`, `test_model.py` and later `test_api.py`. It had never passed.

```
from automated_support_ticket_classification.models.evaluate import main as evaluate_main
from automated_support_ticket_classification.models.train import main as train_main

def test_model_scripts_import_and_define_main():
    assert callable(train_main)
    assert callable(evaluate_main)
```

Applied in commit 52b5872. Worth keeping: it is the only test that imports `evaluate.py`, so it is the sole guard against import-time breakage there.

A2 **FIXED** `models/__init__.py` disguised an artifact directory as a package

Root `models/` exists only to receive `model.joblib`. An `__init__.py` was presumably added to keep the directory in Git, but it also made `import models` resolve to it. Combined with `pythonpath = ["src"]` and `tests/__init__.py`, that is a shadowing hazard, and it is why A1 looked plausible when it was written.

FIX. Use the placeholder that means "keep this directory", not the one that means "this is a package": delete `__init__.py`, keep `.gitkeep`.

Applied in commit 48e07be.

A3 **FIXED** `description` was in the wrong TOML table

Caused by B3. The file read:

```
[build-system]
description = "Production-grade MLOps service that classifies customer support tickets."
requires = ["setuptools>=68"]
```

Consequence: the package had no `Summary` in its metadata, and `setuptools` saw an unexpected key under `[build-system]`.

Applied in commit 5439f1e. Moved under `[project]`; verified by reading the installed metadata, where `Summary` is now populated.

A4 **FIXED** Lint and format were already failing

```
> ruff format --check .    1 file would be reformatted
> ruff check .             Found 5 errors
```

Day 25 adds a CI job running exactly those two commands, so the first push after creating the workflow would have gone red, costing that day to CI debugging rather than CI learning.

Applied across commits 2d10164 and 48e07be. The last four errors were E402s caused by the `sys.path` block in `train.py` and `evaluate.py`, removed once B5 was fixed properly in `config.py`. CI has been green on every run since.

A5 **FIXED** Data CSVs were committed to Git

Three files were tracked: `data/raw/tickets.csv`, `data/processed/train.csv`, `data/processed/test.csv`. DVC will not take ownership of a file Git already tracks. Reproduced in a scratch repo with the Day 16 stage definition:

```
ERROR: failed to reproduce 'generate': output 'data\raw\tickets.csv' is already
tracked by SCM (e.g. Git).
    To stop tracking from Git:
        git rm -r --cached 'data\raw\tickets.csv'
```

This is correct behaviour, not a bug: two version control systems cannot both own one file's history.

Applied in commit 48e07be. Untracked, files untouched on disk. Day 17's `dvc repro` then ran all four stages cleanly.

A6 **FIXED** `metrics/metrics.json` was ignored, and must not be

`.gitignore` contained `metrics/*.json`, confirmed with `git check-ignore -v`. This contradicts two days: Day 12 says commit it so metric changes show up in PR diffs, and Day 16 declares the stage metric with `cache: false`, which is DVC's way of saying Git owns this file rather than DVC. With the ignore rule in place, Day 16 produces a pipeline whose headline output vanishes.

Applied in commit 48e07be. Line removed and the file committed. It has appeared as a tracked modification in every metric-changing commit since, which is the point.

A7 **FIXED** DVC was broken by a transitive dependency

```
ERROR: unexpected error - cannot import name '_DIR_MARK' from
'pathspec.patterns.gitwildmatch'
```

ROOT CAUSE. `dvc 3.55.2` imports that private symbol at `dvc/ignore.py:30`. In **pathspec 0.12.1** it is defined in `pathspec/patterns/gitwildmatch.py` line 21, exactly where DVC looks. In **pathspec 1.1.1**, which pip had resolved, it moved to `pathspec/patterns/gitignore/spec.py` and the old module no longer re-exports it. Both locations were confirmed by reading the package source, not inferred from the traceback.

This is the hidden cost of B4: `requirements.txt` pins `dvc` exactly but says nothing about `pathspec`, so pip silently broke the pinned tool.

Applied in commit 3407a18. `pathspec==0.12.1` installed and pinned with a comment explaining why.

A8 **FIXED** The `iprint` warning on every model fit

```
sklearn\linear_model\_logistic.py:455: OptimizWarning:
Unknown solver options: iprint
```

scikit-learn passes a legacy verbosity option to SciPy's L-BFGS-B solver; SciPy removed it. The boundary was traced by reading each wheel's source rather than guessing:

Package version	Sends or accepts <code>iprint</code> ?
scikit-learn 1.5.2 through 1.9.1	Still sends it. Every release does.
scipy 1.14.1, 1.15.0, 1.16.0, 1.16.2, 1.17.0	Accepts it. No warning.
scipy 1.18.1	Rejects it. Warning on every fit.

Upgrading scikit-learn does **not** help, which is worth knowing because that is the obvious instinct. The change was on the SciPy side.

Applied in commit f14d31e. `scipy<1.18` and `numpy<3` pinned; SciPy is 1.17.1 and the warning is gone.

A9 **FIXED** MLflow's deprecation warnings buried the test output

MLflow 2.18 still uses pydantic v1 style `@validator` decorators, and pydantic 2.9 warns about each one. Not your code, and not silenceable at the source.

```
filterwarnings = [
    "ignore::DeprecationWarning:mlflow.*",
    "ignore:Pydantic V1 style.*:UserWarning",
]
```

Applied in commit 5439f1e. Output went from 23 warnings to 8. The remaining 8 are unrelated deprecations from joblib, sklearn, starlette and pydantic, left visible on purpose: blanket-suppressing everything would defeat the point of having warnings.

A10 **FIXED** Python 3.12 locally, 3.11 everywhere else

The venv was 3.12.10, the guide's Dockerfile `python:3.11-slim`, and its CI workflow pinned 3.11. Three environments, two interpreters, which surfaces as "works on my machine".

Applied in commit f14d31e. Settled on 3.12: both Dockerfile stages use `python:3.12-slim` and the CI workflow pins `"3.12"`. `requires-python = ">=3.11"` stays as a floor, not a target.

Group B · Defects in the guides, by the day they detonate

B1 **FIXED** Day 2 ignores `data/`, then asks Git to add it

Day 2's `.gitignore` contains `data/` and `models/`. The same page then runs `git add .gitignore data models metrics`. Reproduced:

```
exit 1
The following paths are ignored by one of your .gitignore files: data, models
staged: .gitignore, metrics/.gitkeep          # 2 files, not the 5 claimed
```

Git will not descend into an ignored directory, so a negation like `!data/raw/.gitkeep` is unreachable. Ignore the contents:

```
/data/raw/*
/data/processed/*
!/data/raw/.gitkeep
!/data/processed/.gitkeep
/models/*
!/models/.gitkeep
```

B2 **FIXED** Day 3 installs a package that does not exist yet

Day 3 runs `pip install -e .` and promises the import succeeds. It cannot: Day 2 created directories but no `__init__.py`, and the first one appears on Day 5. `find_packages(where="src")` returns `[]`, so the install succeeds and installs nothing. The failure is deferred to Day 5 and shows up as a confusing `ModuleNotFoundError` in a day that has nothing to do with packaging.

B3 **FIXED** Day 3's TOML block reflows into the wrong tables

In the PDF, `name` detaches from its table and floats to the page bottom, and `description` lands next to the `[build-system]` header. Transcribe literally and you get A3.

General rule. When a code block looks like its lines are out of order, they probably are. Section headers always precede their keys. This same mechanism caused D1, which was far more damaging.

B4 **FIXED** The pins do not pin the things that break

Day 3 presents eleven `==` pins and claims this makes builds reproducible. It does not, because none of the three packages that actually broke the environment are named:

Package	Pinned?	What went wrong
<code>pathspec</code>	No	Resolved to 1.1.1 and broke DVC outright (A7).
<code>scipy</code>	No	Resolved to 1.18.1 and warned on every fit (A8).
<code>numpy</code>	No	Floated freely; the next thing to surprise you.

Pinning direct dependencies is not the same as pinning the dependency *graph*.

B5 **FIXED** Day 6's config path is relative to the working directory

`PARAMS_PATH = Path("params.yaml")` resolves against the process working directory, so the loader works only from the repo root. It breaks running `uvicorn` from elsewhere, `pytest` from a subdirectory, or loading config in a container with a different `WORKDIR`.

Fixed independently before the audit, anchoring to `parents[2]`. It is also what makes the Day 23 Dockerfile work: inside the image, `parents[2]` from `/app/src/<pkg>/config.py` lands exactly on `/app`, where `params.yaml` was copied. With the original CWD-relative path that is luck, not design.

B6 **FIXED** Pydantic v2 warns on every `model_` field

```
UserWarning: Field "model_path" in ModelConfig has conflict with protected namespace "model_".
```

Harmless until something runs with `-W error`, at which point config loading raises and every test fails at once with a message unrelated to the test. Fix: `model_config = ConfigDict(protected_namespaces=())` on each model.

B7 **FIXED** Metrics stay as numpy scalars in train.py

`accuracy_score` returns `numpy.float64`. Day 12 casts with `float(...)`; Days 10 and 11 never do. MLflow tolerates the numpy type, so nothing fails today, but serialising those values raises `TypeError: Object of type float64 is not JSON serializable`. Cast where the metric is computed, not where it is written.

B8 **FIXED** The optional-import guard is incomplete

Day 11 writes `if USE_MLFLOW:` then calls `mlflow.start_run()`. Safe at runtime, since the `except ImportError` branch sets `USE_MLFLOW = False` in the same breath, but a type checker cannot see that coupling. Make the invariant explicit: `if USE_MLFLOW and mlflow is not None:`.

B9 **FIXED** Day 16's DVC stages under-declare their dependencies

Each stage lists only its own module, but every module also imports `config.py` and `logger.py`. Edit `config.py` and DVC reports the pipeline up to date, because no declared hash changed. A silent reproducibility hole, and precisely the failure mode Day 16's own "Common mistakes" warns about for params.

Applied in commit 375b2b2.

B10 **FIXED** The API preprocessed differently from training

Training normalises through `clean_text`, which lowercases **and** collapses whitespace. Day 20's handler does only `request.text.strip().lower()`. Two transforms on two sides of one model: train/serve skew.

HONEST IMPACT. Small here, because TF-IDF's default tokenizer splits on non-word characters. **WHY FIX IT.** The bug class is among the most commonly probed in ML engineering interviews, the fix costs one import, and two independent definitions of "clean" guarantee divergence the moment preprocessing grows.

Applied in commit 678770b.

B11 **FIXED** The empty-text guard is unreachable

Day 19 declares `min_length=1`, so pydantic rejects `""` before the handler runs. Day 22's `test_predict_rejects_empty_text` therefore tests pydantic, not your code. The branch is reachable for `" "`, which has length 3 and only the handler's `strip()` catches. Verified directly.

```
def test_predict_rejects_whitespace_only_text():
    response = client.post("/predict", json={"text": " "})
    assert response.status_code == 422
```

Applied in commit 84431e1.

B12 **FIXED** The lean runtime image was not lean

Day 11 justifies the optional-import pattern by saying it lets the file stay lean in Docker. Day 23 then runs `pip install -r requirements.txt`, installing MLflow and DVC with their dependency trees. `DISABLE_MLFLOW=1` stops MLflow being *used*; it does nothing about it being *installed*.

Applied in commit f14d31e. `requirements-serve.txt` added and used by the runtime stage. The `try: import mlflow / except ImportError` path is now genuinely exercised for the first time. Final image 711 MB.

B13 **FIXED** Day 25's CI assumes the venv is `.venv`

Ruff auto-excludes conventional virtual environment names. This project's is `ML0ps`, which ruff has never heard of, so `ruff check .` walks thousands of site-packages files and reports other people's style violations as yours.

Worth doing next time: name the venv `.venv`. Every tool in the Python ecosystem special-cases that name. The same line is needed in `.dockerignore`, or the whole environment ships as build context.

B14 **FIXED** Day 15 commits a Windows path into `.dvc/config`

`dvc remote add -d localremote ../dvc-storage` writes a backslash path into a file that is committed to Git. It works locally and is meaningless on a Linux runner. Use a forward slash, which Windows accepts perfectly well.

Applied in commit 48e39b9.

B15 **FIXED** The badge URLs do not match the real repository

Two problems at once. The Automated-Support-Ticket PDF hard-codes the handle `raviteja311`; the Ticket-Triage PDF leaves literal `<user>` placeholders. Worse, **both use the all-lowercase repo name**, while the actual repository is `Automated-Support-Ticket-Classification`. Copying either verbatim yields a permanently broken badge image on the README a recruiter reads first.

Applied in commit 7faf051. Real repo name used and confirmed to return HTTP 200 before committing.

B16 **FIXED** The guide's API code fails the guide's own lint rules

```
app.py:40:43: B905 `zip()` without an explicit `strict=` parameter
```

Day 20 writes `zip(model.classes_, proba)`. Day 4 configures ruff with `select = ["E", "F", "I", "UP", "B"]`, and `B` is bugbear, which includes B905. Following the guide literally gives a red CI job on Day 25, in a file you did not write.

```
scores = {cls: float(p) for cls, p in zip(model.classes_, proba, strict=True)}
```

Applied in commit 678770b. Adding the parameter is also semantically right: the sequences must be the same length, and silent truncation is the worst outcome.

B17 **FIXED** The smoke test cannot survive a cold start

Day 29's workflow issues a single `curl` against `/health` and fails if it is not 200. Day 28 of the same guide states that free instances sleep when idle and the first request after a nap is slow. Those two facts are incompatible: the daily run would fail most mornings, and a monitor that cries wolf is a monitor you stop reading.

```
for attempt in $(seq 1 10); do
  code=$(curl -s -m 30 -o /dev/null -w "%{http_code}" "$SERVICE_URL/health" || echo "000")
  [ "$code" = "200" ] && exit 0
  sleep 15
done
exit 1
```

Applied in commit a00a543. Also guards on `SERVICE_URL` being set, and adds a `/predict` check asserting one of the five known labels, which proves the model artifact loaded rather than merely that the process is listening.

B18 **FIXED** Day 3 declares a dependency the project never uses

SYMPTOM. None. It is a correctness-of-documentation defect, found by auditing declared dependencies against actual imports.

ROOT CAUSE. Day 3's `requirements.txt` lists `pydantic-settings==2.6.1`. Nothing in `src/` or `tests/` imports it, at any point in the 30 days. Grepping the finished project returns zero matches.

```
pydantic_settings          0 import(s)
```

WHY IT MATTERS. A requirements file is a statement about what the project needs. Declaring a package nobody imports is a small lie that misleads anyone auditing the dependency surface, and it is the kind of thing that accumulates until no one trusts the file.

FIX. Remove the line. There is no functional change: `gto`, which arrives transitively via DVC, requires it, so it still gets installed. FastAPI only wants it under the `[all]` extra, which this project does not use.

Applied in commit db6aed4. Verified afterwards that the suite, the lint, the pipeline and the Docker build all still pass, the last of which independently confirms the serving path is unaffected.

Group C · Documentation that is simply wrong

These are statements in the source PDFs. Nothing in a repository can fix them; they are listed so the stated expected output does not confuse you.

ID	Where	What it claims	What actually happens
C1	Day 4	1 file reformatted, 4 files left unchanged	At Day 4 the repo contains no Python files; the first arrives on Day 5. Expect 0 reformatted.
C2	Day 2	5 files changed, 40 insertions(+)	Two files stage and the <code>git add</code> exits 1. See B1.
C3	Day 2	<code>.github/workflows/</code> is part of the skeleton	Git cannot track an empty directory and the day's <code>git add</code> omits it, so it never reaches the remote. Harmless: Day 25 recreates it with a file in it.
C4	all	Two separate guides	One guide, two package names. FIXED by consolidating into this edition.

Group D · Defects found while building

Not in the original audit, because it was not visible until the DVC pipeline existed. It is the best interview material in the project, because it was found by reading output rather than by being told.

D1 **FIXED** The dataset had four classes, not five

HOW IT SURFACED. Reading `dvc metrics show` on Day 17. The per-class table listed `account`, `billing`, `general` and `technical`, with supports summing to exactly 600. `shipping` was absent.

ROOT CAUSE. A transcription error in `TEMPLATES` in `data/generate.py`, made when the generator was copied from the PDF. The `"shipping": [` key sits at the bottom of page 22 and its five entries resume on page 23, with an unrelated file-path caption interleaved. Two things went wrong at that page break:

1. `shipping` was dropped entirely, leaving four classes.
2. The five `technical` templates were filed under `account`, giving it ten templates, half describing technical problems. Five templates appearing nowhere in either PDF had been written to fill the resulting gap.

So half of every `account` row was a technical ticket wearing the wrong label. The model was not underperforming; it was being asked to learn a contradiction. A quieter third symptom confirmed it: the `order` entry in `FILLERS` was dead code, because only the missing shipping templates used `{order}`.

EVIDENCE BEFORE THE FIX. `account` had the worst recall of any class at 0.869, and `technical` the worst precision at 0.905, which is precisely the signature of one class's examples being predicted as another.

Metric	Before (4 classes)	After (5 classes)
accuracy	0.9217	0.9300
macro F1	0.9212	0.9297
account recall	0.869	0.920

Accuracy rose *despite* the task gaining a class, because removing the contradictory labels mattered more than the extra class cost. `account` recall becoming unremarkable among its peers is exactly what you would predict if contamination had been the cause.

INDEPENDENT CONFIRMATION. 0.9300 and 0.9297 match the figures the guide prints as expected output for Day 10, so the dataset now matches the one the guide intends.

WHY NOTHING CAUGHT IT. The training logs reported a healthy 0.92. The unit tests assert row counts and column names, not the label set. Only per-class metrics laid side by side made it visible, which is the whole argument for Day 12 existing.

Applied in commit ee2a709. A guard test, `test_generate_covers_all_five_categories`, pins the label set so this cannot regress silently. Written up as E2 in `docs/experiments.md`.

The pattern worth carrying forward. D1, A3 and B3 share one cause: a PDF page break scrambling a code block during transcription. In A3 it misplaced a TOML key and was harmless. In D1 it dropped a class and mislabelled a quarter of the dataset. When a code block spans a page boundary, check it against the surrounding structure rather than trusting the reading order.

Part 2 · Repair sprint

All nine repairs are complete. The sequence is kept so the reasoning survives and so the project can be rebuilt from scratch.

Run from the repo root with the environment active. Order matters: Repair 4 untracks files, so do Repair 3 first or the next `git add -A` puts them straight back.

DONE Repair 1 · Fix the broken environment (A7, A8, B4)

```
pip install "pathspec==0.12.1" "scipy<1.18"
dvc --version          # 3.55.2 with no traceback
pytest                 # no iprint OptimizeWarning
```

Then record the pins in `requirements.txt`: `pathspec==0.12.1`, `scipy<1.18`, `numpy<3`.

DONE Repair 2 · Fix pyproject.toml (A3, A9, B13)

Move `description` to `[project]`, add the `filterwarnings` block, keep the `MLOps` ruff exclusion and comment it. Then `pip install -e .` so the corrected metadata takes effect.

DONE Repair 3 · Fix .gitignore (A6, B1)

Replace the artifact block with the contents-not-directories form shown in B1, and delete the `metrics/*.json` line. `metrics/metrics.json` must be Git-tracked: Day 12 wants it in PR diffs and Day 16 declares it `cache: false`.

DONE Repair 4 · Untrack the data artifacts (A5)

```
git rm -r --cached data/raw/tickets.csv data/processed/train.csv data/processed/test.csv
```

The files stay on disk; Git stops tracking them. This is what unblocks Day 17.

DONE Repair 5 · Start tracking metrics.json (A6)

```
git add metrics/metrics.json
git check-ignore -v metrics/metrics.json # must print nothing
```

Once `metrics.json` is tracked, `metrics/.gitkeep` becomes redundant and can go: the directory is no longer empty, and `evaluate.py` recreates it with `mkdir(parents=True, exist_ok=True)` regardless. The placeholders under `data/` and `models/` stay, because everything else there is gitignored.

DONE Repair 6 · Demote models/ to a plain directory (A2)

```
git rm --cached models/__init__.py
Remove-Item models\__init__.py
if (-not (Test-Path models\.gitkeep)) { New-Item -ItemType File models\.gitkeep }
```

DONE Repair 7 · Clear the lint and format backlog (A4)

```
ruff format .
ruff check . --fix
ruff check . # All checks passed!
```

Do this before Day 25, not during it. Arriving at the CI day with a clean tree is what makes the workflow green on its first run, which is the entire point of that exercise.

DONE Repair 8 · Commit

On this repo it landed across several commits: `3407a18` for the pathspec pin, `48e07be` for the ignore rules and the `sys.path` removal, `5439f1e` for A3 and A9, and `db6aed4` for B18.

DONE Repair 9 · Verify

Command	Required result
<code>pytest</code>	<code>13 passed</code> at this point, no <code>OptimizeWarning</code>
<code>ruff check .</code>	<code>All checks passed!</code>
<code>ruff format --check .</code>	no file would be reformatted
<code>dvc --version</code>	<code>3.55.2</code> , no traceback
<code>git ls-files data models</code>	only <code>.gitkeep</code> entries

What this repair is worth in an interview

- **Dependency resolution.** "My pinned DVC broke because an unpinned transitive dependency moved a private symbol between minor versions. I traced it to the import in `dvc/ignore.py`, found the symbol had relocated in pathspec 1.x, and pinned it. Pinning direct dependencies is not the same as pinning the graph."
- **Tooling boundaries.** "DVC refused to take over files Git already tracked. That is correct behaviour: two version control systems cannot both own one file's history."
- **Reading the right layer.** "A scikit-learn warning looked like an ML problem. I checked the source of every release from 1.5 to 1.9 and they all still send that option; the change was on the SciPy side. Upgrading scikit-learn would not have fixed it."
- **Dependency hygiene.** "I audited declared dependencies against actual imports and found one the project never used. Removing it changed nothing functionally, which is the point: a requirements file should describe what the code needs, not what a tutorial happened to list."

Part 3 · Days 14 to 30, corrected

Each day keeps the original shape. A **Corrected** note marks where this edition departs from the source PDFs, and a **Completed** banner records what actually happened when the day was built, with its commit. Days 1 to 13 are unchanged apart from the fixes covered in Parts 1 and 2.

Day 14 · Run a real experiment

Modeling · DONE

Completed, commit [95276f9](#) . Both configurations were actually run. Unigrams and bigrams scored identically: accuracy 0.9217, macro F1 0.9212.

OBJECTIVE. Change one hyperparameter, retrain, compare in MLflow, record the decision.

WHY IT MATTERS. The day the tracking setup pays for itself. Form a hypothesis, change exactly one variable, rerun, compare. Then write down the outcome and commit the decision, not just the code. An experiment you did not record is one you will repeat.

```
# set ngram_max: 2 in params.yaml, then:
python -m automated_support_ticket_classification.models.train
mlflow ui                # compare the two runs
# decide, write it up, set ngram_max back to the winner
```

RESULT. Identical to four decimal places, so `ngram_max` stays at 1. When scores tie the tie breaks on cost: bigrams inflate the vocabulary for no gain, meaning a larger vectoriser, artifact and image to make the same predictions.

Record the measured numbers, not the guide's placeholders. A portfolio experiments log with invented figures is worse than none. Note also that these numbers predate the D1 data fix, so they are not comparable to anything later; `docs/experiments.md` carries that caveat explicitly.

COMMIT. `chore: experiment with bigram features and record results`

Completed, commit 48e39b9 . DVC initialised, `localremote` set, URL stored as `../../dvc-storage` with forward slashes.

WHY IT MATTERS. Git keeps the full history of every version of every binary, so repos bloat and clones crawl. DVC keeps a tiny pointer in Git and puts the bytes in a remote. This is how a team shares the exact dataset and model behind a given commit.

Corrected. Confirm Repair 1 and Repair 4 first, or nothing on this day works. Use a forward slash for the remote path (B14) so the committed `.dvc/config` is not Windows-only.

```
# Preflight, both must be clean
dvc --version           # 3.55.2, no traceback (Repair 1)
git ls-files data       # only .gitkeep entries (Repair 4)

dvc init
mkdir ../dvc-storage
dvc remote add -d localremote ../dvc-storage
git add .dvc .dvcignore .gitignore
```

COMMIT. `chore: initialize DVC with a local remote`

Completed, commit `375b2b2` . `dvc dag` renders the correct four-node graph.

Corrected. Every stage also depends on `config.py` and `logger.py` (B9). Without those, editing shared code leaves DVC convinced the pipeline is up to date.

```
stages:
  generate:
    cmd: python -m automated_support_ticket_classification.data.generate
    deps:
      - src/automated_support_ticket_classification/data/generate.py
      - src/automated_support_ticket_classification/config.py
      - src/automated_support_ticket_classification/logger.py
    params: [data.n_samples, data.random_state]
    outs:
      - data/raw/tickets.csv

  preprocess:
    cmd: python -m automated_support_ticket_classification.data.preprocess
    deps:
      - src/automated_support_ticket_classification/data/preprocess.py
      - src/automated_support_ticket_classification/config.py
      - src/automated_support_ticket_classification/logger.py
      - data/raw/tickets.csv
    params: [data.test_size, data.random_state]
    outs:
      - data/processed/train.csv
      - data/processed/test.csv

  train:
    cmd: python -m automated_support_ticket_classification.models.train
    deps:
      - src/automated_support_ticket_classification/models/train.py
      - src/automated_support_ticket_classification/config.py
      - src/automated_support_ticket_classification/logger.py
      - data/processed/train.csv
      - data/processed/test.csv
    params: [model.max_features, model.ngram_max, model.C, model.max_iter]
    outs:
      - models/model.joblib

  evaluate:
    cmd: python -m automated_support_ticket_classification.models.evaluate
    deps:
      - src/automated_support_ticket_classification/models/evaluate.py
      - src/automated_support_ticket_classification/config.py
      - src/automated_support_ticket_classification/logger.py
      - models/model.joblib
      - data/processed/test.csv
    metrics:
      - metrics/metrics.json:
          cache: false
```

The `params: [a, b]` inline form is equivalent to the block list and is used here only to fit the page. Either is valid YAML.

CONCEPT TO KEEP. `cache: false` means Git owns the metric file rather than DVC. That is exactly why Repair 3 removed `metrics/*.json` from `.gitignore`: an ignored file cannot appear in a PR diff, which is the only reason to declare it this way.

COMMIT. feat: define DVC pipeline (generate to evaluate)

Completed, commit `55f45b0` . All four stages ran and `metrics.json` came out byte-identical to the committed copy, confirming the seeded generator and split are deterministic. **Reading this day's output is how D1 was found.**

```
dvc repro
dvc metrics show
git add dvc.yaml dvc.lock metrics/metrics.json
```

If `dvc repro` fails with `ModuleNotFoundError` , DVC spawned a different `python` . `dvc.yaml` hardcodes the interpreter name, so run it from the activated environment or put the venv first on PATH. This is not in either PDF.

If it fails with `already tracked by SCM` , Repair 4 was skipped. DVC is telling the truth: two version control systems cannot both own one file.

DO NOT SKIM `DVC METRICS SHOW` . The per-class table is the only place a missing class is visible. Count the classes and check the supports sum to your test-set size.

COMMIT. `chore: run pipeline and lock reproducible outputs`

Completed, no commit. 4 files pushed; cache and remote in sync. This day changes nothing in the repository, so there is nothing to commit. The guide says these are genuine commits and never filler, so do not manufacture an empty one.

```
dvc push
dvc status
dvc status -c      # compare workspace against the remote
```

COMMON MISTAKES. Expecting `git push` to move data: it does not, ever. Using a local remote and wondering why CI cannot pull: a folder on your laptop is not reachable from a GitHub runner. For a real project point the remote at S3 or GCS, which is a one-command change.

COMMIT. none, by design.

Day 19 · API request and response schemas

Serving · DONE

Completed, commit [84665b7](#) . Verified directly: pydantic rejects `""` but admits `" "` , which is exactly the asymmetry B11 describes.

```
from pydantic import BaseModel, Field

class TicketRequest(BaseModel):
    text: str = Field(..., min_length=1, examples=["I was charged twice this month"])

class TicketResponse(BaseModel):
    label: str
    confidence: float
    all_scores: dict[str, float]
```

Know what this does and does not catch. `min_length=1` rejects the empty string. It admits three spaces. Day 20 handles that; Day 22 tests it.

COMMIT. `feat: add API request/response schemas`

Completed, commit [678770b](#) . Verified against the running service: a billing ticket scored 0.938 and all five classes appear in `all_scores` .

Corrected, two changes. The handler reuses `clean_text` rather than its own `.strip().lower()` (B10). And `zip()` takes `strict=True` (B16): the guide's version trips B905 from the ruleset the guide itself enables on Day 4.

```
from functools import lru_cache

import joblib
from fastapi import FastAPI, HTTPException

from automated_support_ticket_classification.api.schemas import TicketRequest, TicketResponse
from automated_support_ticket_classification.config import load_config
from automated_support_ticket_classification.data.preprocess import clean_text
from automated_support_ticket_classification.logger import get_logger

logger = get_logger(__name__)

app = FastAPI(title="Automated Support Ticket Classification API", version="1.0.0")

@lru_cache
def get_model():
    cfg = load_config()
    logger.info("Loading model from %s", cfg.model.model_path)
    return joblib.load(cfg.model.model_path)

@app.get("/health")
def health() -> dict:
    return {"status": "ok"}

@app.post("/predict", response_model=TicketResponse)
def predict(request: TicketRequest) -> TicketResponse:
    # Same normalisation as training. Reuse it, never restate it.
    text = clean_text(request.text)
    if not text:
        # Reachable for whitespace-only input, which passes min_length=1.
        raise HTTPException(status_code=422, detail="text must not be empty")

    model = get_model()
    proba = model.predict_proba([text])[0]
    # strict=True: the two sequences must be the same length.
    scores = {cls: float(p) for cls, p in zip(model.classes_, proba, strict=True)}
    best = max(scores, key=scores.get)
    return TicketResponse(label=best, confidence=scores[best], all_scores=scores)
```

COMMON MISTAKES. Loading the model at import, so a missing file stops the app starting. Reloading per request, so the API is needlessly slow. A lazy getter behind `lru_cache` solves both.

COMMIT. `feat: add FastAPI service with /health and /predict`

Completed, commit `1c8af5d` .

```
@app.get("/")
def root() -> dict:
    return {
        "service": "automated-support-ticket-classification",
        "docs": "/docs",
        "health": "/health",
    }
```

```
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/predict `
-ContentType "application/json" -Body '{"text":"my invoice is wrong"}'
```

The root endpoint returns JSON, not a page. Browsing to `http://127.0.0.1:8000/` shows a small pointer object. The interactive UI is at `/docs` . Your browser will also request `/favicon.ico` and get a 404 every time; that is normal for an API and not an error.

COMMON MISTAKES. Reaching for `curl` in PowerShell and losing ten minutes to quoting. `Invoke-RestMethod` is native and parses the JSON.

COMMIT. `feat: add root endpoint and document local run`

Completed, commit [84431e1](#) . Suite is 13 passed. Verified the fixture by moving `model.joblib` aside and re-running: the API tests passed against a model built from nothing, which is the CI condition.

Corrected, three tests beyond the guide. `test_predict_rejects_whitespace_only_text` (B11) exercises the handler's guard, which the empty-string test never reaches.

`test_predict_scores_cover_every_class_and_sum_to_one` would have caught D1.

`test_root_points_at_the_docs` covers the Day 21 endpoint the guide adds but never tests.

```
@pytest.fixture(scope="session", autouse=True)
def ensure_model():
    """Guarantee a model file exists so API tests can load it (local and CI).

    The label set matches the real pipeline's five classes; a mismatch
    would mean the API tests exercise a model production cannot produce.
    """
    cfg = load_config()
    path = Path(cfg.model.model_path)
    if not path.exists():
        df = pd.DataFrame({...}) # 10 rows per class, all five classes
        pipe = build_pipeline(cfg)
        pipe.fit(df["text"], df["label"])
        path.parent.mkdir(parents=True, exist_ok=True)
        joblib.dump(pipe, path)
    yield

def test_predict_scores_cover_every_class_and_sum_to_one():
    response = client.post("/predict", json={"text": "my package arrived damaged"})
    scores = response.json()["all_scores"]
    assert set(scores) == {"billing", "technical", "account", "shipping", "general"}
    assert abs(sum(scores.values()) - 1.0) < 1e-6
```

COMMON MISTAKES. API tests failing in CI because no model exists; the fixture is the answer. Forgetting `httpx` in dev deps, which `TestClient` needs.

COMMIT. `test: add API tests with a model fixture`

Completed, commit `f14d31e` . Image builds at 711 MB with a 3.26 kB build context. The model bakes in at accuracy 0.9300, identical to the local pipeline, which confirms the seeded generator is reproducible across machine boundaries.

Corrected, two changes. The runtime installs `requirements-serve.txt` , excluding MLflow and DVC (B12). The base image is 3.12 in both stages to match the local interpreter (A10).

```
# ----- Stage 1: build dependencies -----
FROM python:3.12-slim AS builder
WORKDIR /app
ENV PIP_NO_CACHE_DIR=1 PYTHONDONTWRITEBYTECODE=1
COPY requirements-serve.txt .
RUN pip install --prefix=/install -r requirements-serve.txt

# ----- Stage 2: lean runtime -----
FROM python:3.12-slim AS runtime
WORKDIR /app
ENV PYTHONUNBUFFERED=1 PYTHONPATH=/app/src DISABLE_MLFLOW=1
COPY --from=builder /install /usr/local
COPY src/ ./src/
COPY params.yaml ./

RUN python -m automated_support_ticket_classification.data.generate \
    && python -m automated_support_ticket_classification.data.preprocess \
    && python -m automated_support_ticket_classification.models.train

EXPOSE 8000
CMD ["sh", "-c", "uvicorn automated_support_ticket_classification.api.app:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

`.dockerignore` must include `MLOps` as well as `.venv` , or the whole environment ships as build context. Also exclude `data/` , `models/` , `metrics/` , `mlruns/` , `tests/` , `docs/` , `.git` , `.github` and the tool caches.

Why this build works depends on the B5 fix. Inside the image `config.py` sits at `/app/src/<pkg>/config.py` , so `parents[2]` resolves to `/app` where `params.yaml` was copied. With the original CWD-relative path that is luck, and it stops being lucky the moment `WORKDIR` changes.

Docker is optional for this project. `uvicorn` alone runs and exercises the whole service. The image exists to prove the build is portable and to give CI something to verify on a clean machine.

COMMIT. `build: add multi-stage Dockerfile`

Completed, commit 9f972a7 . Container serves `/` , `/health` , `/docs` and `/predict` . A shipping ticket classified correctly at 0.934. The `$PORT` override was verified with `-e PORT=9001` .

```
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DISABLE_MLFLOW=1
```

Give uvicorn a few seconds. The container reports `Up` before the server has bound its port, so an immediate `curl` returns empty. Poll rather than assuming a single request will land.

COMMON MISTAKES. Omitting `-p 8000:8000` , so the container runs and nothing can reach it. Port 8000 taken: map a different host port.

COMMIT. `build:` add `docker-compose` for local run

Completed, commit [72367d1](#) . Green on the first push. That held only because the B16 fix landed on Day 20 and the lint backlog was cleared in Repair 7.

Corrected. Python 3.12 to match the Dockerfile and venv (A10); `ruff format --check` runs before `ruff check` so the cheaper, more common failure reports first; `cache: pip` added; actions bumped to `checkout@v5` and `setup-python@v6` , since the guide's v4/v5 pair triggers a Node 20 deprecation annotation on every run.

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-python@v6
        with:
          python-version: "3.12"
          cache: pip
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements-dev.txt
          pip install -e .
      - name: Format check
        run: ruff format --check .
      - name: Lint
        run: ruff check .
      - name: Tests
        run: pytest
```

Simulate the runner before you push. Move `model.joblib` aside and run `pytest` . If the fixture is right, all 13 pass against a model built from nothing. Green locally is the only reliable predictor of green in CI, which is this day's real lesson.

COMMIT. `ci: add lint and test workflow`

Completed, commit `7faf051` . Both jobs green; the docker job finished in 40 seconds.

Corrected, two changes. The job also **smoke-tests the image**: it starts the container and polls `/health` . Building proves the image compiles; running it proves it serves traffic, which is the claim you actually want. And the badge URL uses the **real repo name** (B15): both PDFs use an all-lowercase name that does not match, giving a permanently broken badge.

```
docker-build:
  runs-on: ubuntu-latest
  needs: lint-and-test
  steps:
    - uses: actions/checkout@v5
    - name: Build Docker image
      run: docker build -t automated-support-ticket-classification:ci .
    - name: Smoke test the image
      run: |
        docker run -d --name ci-api -p 8000:8000 automated-support-ticket-classification:ci
        for i in $(seq 1 30); do
          curl -sf http://127.0.0.1:8000/health && break
          sleep 2
        done
        curl -sf http://127.0.0.1:8000/health | grep -q '"status":"ok"'
        docker rm -f ci-api
```

INTERVIEW ANGLE. Order jobs with `needs:` so cheap checks run first and expensive ones only on already-passing code. Lint takes seconds; this build trains a model.

COMMIT. `ci: build docker image in pipeline and add status badge`

Completed, commit `a368df5` . `/metrics` returns 200 with real counters against the locally running service, and CI confirms the instrumented image builds and serves.

Verified end to end. With both services running, the Prometheus target reports `health: UP` against `http://api:8000/metrics` with no error, and querying Prometheus returns the stored counters: `/predict` at exactly the 5 requests sent, `/health` at 3, and `/metrics` rising each interval as Prometheus scrapes. 12 duration histogram buckets confirm all three RED signals.

If Docker will not start on Windows, check `wsl -l -v`. A `docker-desktop` distro stuck at `Stopped` is the usual cause. Kill the `Docker Desktop` and `com.docker.backend` processes, run `wsl --shutdown`, then relaunch. The engine came back in about 30 seconds.

```
from prometheus_fastapi_instrumentator import Instrumentator

app = FastAPI(title="Automated Support Ticket Classification API", version="1.0.0")
Instrumentator().instrument(app).expose(app)
```

```
# prometheus.yml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: automated-support-ticket-classification
    metrics_path: /metrics
    static_configs:
      - targets: ["api:8000"]
```

Add a `prometheus` service to compose using `prom/prometheus:latest`, publishing 9090, mounting `./prometheus.yml` read-only, with `depends_on: [api]`.

COMMON MISTAKES. Expecting metrics before any traffic: counters start at zero, so send requests first. Targeting `localhost` instead of the service name `api`, which inside the compose network means the Prometheus container itself.

INTERVIEW ANGLE. Track the RED signals: Rate, Errors, Duration. For an ML service you would add input and prediction drift on top, which is a different job from service health and needs different tooling.

COMMIT. `feat: expose Prometheus metrics and add monitoring stack`

Blueprint committed, deploy deliberately not performed, commit `2f00299` . This project runs locally by design, so no Render service was created. The file below is complete and correct; deploying is a decision, not a missing step.

```
services:
- type: web
  name: automated-support-ticket-classification
  runtime: docker
  dockerfilePath: ./Dockerfile
  plan: free
  healthCheckPath: /health
  autoDeploy: true
  envVars:
  - key: DISABLE_MLFLOW
    value: "1"
```

WHAT THE CONFIG BUYS YOU. `healthCheckPath` gates the rollout: Render polls `/health` and only routes traffic once an instance answers, so a broken deploy never receives requests. `autoDeploy` makes every merge to main ship, which is what gives the Day 29 smoke test something to verify.

IF YOU DO DEPLOY. Sign in, New then Web Service, connect the repo, confirm Runtime Docker and Health Check Path `/health`, create, then open the URL with `/docs` appended. Expect a slow first build: the image is 711 MB and trains a model.

Three details that make this work at all, all decided earlier: the model is baked into the image so there is no external storage to configure; the container reads `$PORT` rather than hardcoding 8000; and it binds `0.0.0.0`, not `127.0.0.1`. That last one is the single most common reason a container works locally and returns nothing when deployed.

Also confirm the current free tier still exists and what its limits are. That detail changes between a guide being written and you following it.

COMMIT. `deploy: add Render blueprint`

Workflow committed and manual-only, commits `a00a543` and `b7d5310` . Triggering it with no secret set failed at the first step with a readable message, which is the guard working as designed. The daily schedule is commented out because this project is not deployed: on a timer it would fail every morning and leave the Actions tab permanently red, which reads as a broken project rather than an undeployed one.

Corrected, three changes (B17). Guards on `SERVICE_URL` being set. Retries `/health` ten times with a 15s gap, because a sleeping free instance can take over a minute and the guide's single request would fail most mornings. Also checks `/predict` returns one of the five known labels, because `/health` only proves the process is listening.

```
name: Smoke Test

on:
  # Manual only. Uncomment the schedule if you ever deploy.
  workflow_dispatch:
  # schedule:
  #   - cron: "0 8 * * *"

jobs:
  smoke:
    runs-on: ubuntu-latest
    steps:
      - name: Check the service URL is configured
        env:
          SERVICE_URL: ${ secrets.SERVICE_URL }
        run: |
          if [ -z "$SERVICE_URL" ]; then
            echo "::error::SERVICE_URL secret is not set."
            exit 1
          fi

      - name: Check live /health
        env:
          SERVICE_URL: ${ secrets.SERVICE_URL }
        run: |
          for attempt in $(seq 1 10); do
            code=$(curl -s -m 30 -o /dev/null -w "%{http_code}" "$SERVICE_URL/health" || echo "000")
            echo "attempt $attempt: HTTP $code"
            [ "$code" = "200" ] && exit 0
            sleep 15
          done
          exit 1
```

INTERVIEW ANGLE. Continuous delivery keeps main always releasable with a manual approval. Continuous deployment ships every passing merge automatically. Know your rollback path either way.

COMMIT. `ci: add scheduled smoke test against live service`

Completed, commit `d811344` , tagged and released as `v1.0.0` . README checked for leftover placeholders, stray non-ASCII and broken relative links; all clean.

Corrected, two departures, both about not overclaiming. No live demo link: the guide puts `<YOUR_RENDER_URL>/docs` at the top, which ships a dead link when nothing is deployed, precisely the mistake the guide's own Day 30 warns about. **The tag does not say "First production release"**, because nothing is in production; it says feature-complete.

README STRUCTURE. Lead with what it is, badges, a one-line statement that it runs locally, architecture at a glance, current measured scores, quickstart, API usage with a worked example per class, pipeline reproduction, tests, Docker, layout, experiments, licence. Optimised for a 30-second skim.

Say plainly that it runs locally. A reader who sees deployment files but no live URL will assume the deploy failed. One sentence near the top turns an apparent gap into an obvious scope decision, and the same sentence belongs in the release notes.

```
git add README.md CONTRIBUTING.md
git commit -m "docs: finalize README and add contributing guide"
git push
git tag -a v1.0.0 -m "v1.0.0 - feature-complete MLOps service"
git push origin v1.0.0
gh release create v1.0.0 --verify-tag --notes-file notes.md
```

COMMON MISTAKES. Leaving placeholders in badges and links. Claiming a live demo you do not have. Writing a README that describes the code rather than the product.

COMMIT. `docs: finalize README and add contributing guide`

Part 4 · Reference files

Final, correct contents. These supersede the equivalent blocks in both PDFs.

pyproject.toml (A3, A9, B3, B13)

```
[project]
name = "automated_support_ticket_classification"
version = "0.1.0"
description = "Production-grade MLOps service that classifies customer support tickets."
requires-python = ">=3.11"

[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
where = ["src"]

[tool.ruff]
line-length = 100
target-version = "py311"
# MLOps is this project's virtual environment. Ruff only auto-excludes
# conventional names such as .venv, so it must be listed explicitly.
exclude = [".git", ".venv", "venv", "MLOps", "__pycache__", ".pytest_cache", ".mypy_cache"]

[tool.ruff.lint]
# E/W pycodestyle, F pyflakes, I isort, UP pyupgrade, B bugbear
select = ["E", "F", "I", "UP", "B"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]
addopts = "-q"
filterwarnings = [
    "ignore::DeprecationWarning:mlflow.*",
    "ignore:Pydantic V1 style.*:UserWarning",
]
```

config.py (B5, B6)

```
from pathlib import Path

import yaml
from pydantic import BaseModel, ConfigDict

# Anchor to the package, never to the working directory.
# config.py -> .../src/<pkg>/ -> .../src/ -> project root
PROJECT_ROOT = Path(__file__).resolve().parents[2]
PARAMS_PATH = PROJECT_ROOT / "params.yaml"

def _resolve_project_path(value: str | Path) -> str:
    path = Path(value)
    return str(path) if path.is_absolute() else str((PROJECT_ROOT / path).resolve())

class ModelConfig(BaseModel):
    model_config = ConfigDict(protected_namespaces=())
    max_features: int
    ngram_max: int
    C: float
    max_iter: int
    model_path: str

# DataConfig, EvaluateConfig and Config follow the same shape, each with
# model_config = ConfigDict(protected_namespaces=()).

def load_config(path: str | Path = PARAMS_PATH) -> Config:
    config_path = Path(path)
    if not config_path.is_absolute():
        config_path = PROJECT_ROOT / config_path
    with open(config_path, encoding="utf-8") as f:
        raw = yaml.safe_load(f)
    cfg = Config(**raw)
    cfg.data.raw_path = _resolve_project_path(cfg.data.raw_path)
    cfg.data.processed_dir = _resolve_project_path(cfg.data.processed_dir)
    cfg.model.model_path = _resolve_project_path(cfg.model.model_path)
    cfg.evaluate.metrics_path = _resolve_project_path(cfg.evaluate.metrics_path)
    return cfg
```

requirements.txt (A7, A8, B4, B18)

```
scikit-learn==1.5.2
pandas==2.2.3
joblib==1.4.2
PyYAML==6.0.2
pydantic==2.9.2
fastapi==0.115.5
uvicorn[standard]==0.32.1
mlflow==2.18.0
dvc==3.55.2
prometheus-fastapi-instrumentator==7.0.0

# Transitive pins. Without these, pip resolves versions that break the
# packages pinned above. Both were found the hard way.
#  pathspec 1.x moved _DIR_MARK; dvc 3.55.2 imports it from the old path.
#  scipy 1.18 dropped the lbfgs iprint option that every sklearn release
#  still sends, producing an OptimizeWarning on every fit.
pathspec==0.12.1
scipy<1.18
numpy<3
```

Ten runtime packages, three transitive pins. The guide's `pydantic-settings` is deliberately absent: nothing imports it (B18).

requirements-serve.txt (B12, new)

```
# Runtime only. No mlflow, no dvc: neither is used at serving time, and
# together they add a large dependency tree to a "lean" image.
scikit-learn==1.5.2
scipy<1.18
numpy<3
pandas==2.2.3
joblib==1.4.2
PyYAML==6.0.2
pydantic==2.9.2
fastapi==0.115.5
uvicorn[standard]==0.32.1
prometheus-fastapi-instrumentator==7.0.0
```

.gitignore, the project-specific block (A5, A6, B1)

```
# Virtual environment (this project's is named MLOps, not .venv)
MLOps/
.venv/
venv/

mlruns/
mlartifacts/

# Data and model artifacts: DVC-tracked, deliberately not in Git.
# Ignore the contents rather than the directories, so the .gitkeep
# placeholders stay visible. Git will not descend into an ignored
# directory, so a negation inside one is unreachable.
/data/raw/*
/data/processed/*
!/data/raw/.gitkeep
!/data/processed/.gitkeep
/models/*
!/models/.gitkeep

# metrics/metrics.json is deliberately NOT ignored: Day 12 wants it in
# PR diffs, and Day 16 declares it cache: false, which means Git owns it.

/.dvc/cache
/.dvc/tmp
```

Three `.gitkeep` placeholders, not four. `metrics/` needs none once `metrics.json` is tracked.

Part 5 · Troubleshooting

The original table, plus every failure actually encountered. New rows carry the errata ID.

Symptom	Likely cause	Fix
<code>FileNotFoundError:</code> <code>...\models\train.py</code>	A test points at a root-level script that never existed A1	Import the package; do not load files by path.
<code>cannot import name</code> <code>'_DIR_MARK'</code>	pathspec 1.x with dvc 3.55.2 A7	<code>pip install "pathspec==0.12.1"</code>
<code>output '...' is already</code> <code>tracked by SCM</code>	DVC cannot own a file Git tracks A5	<code>git rm -r --cached</code> then fix <code>.gitignore</code> .
<code>OptimizeWarning: Unknown</code> <code>solver options: iprint</code>	scipy 1.18 dropped an option sklearn still sends A8	<code>pip install "scipy<1.18"</code> . Upgrading scikit-learn does not help.
Wall of <code>PydanticDeprecatedSince20</code>	MLflow 2.18 uses pydantic v1 validators A9	Add <code>filterwarnings</code> to <code>[tool.pytest.ini_options]</code> .
Ruff reports hundreds of errors in files you never wrote	It is linting your virtual environment B13	Add <code>MLOps</code> to <code>[tool.ruff] exclude</code> .
<code>B905 zip() without an</code> <code>explicit strict=</code>	The guide's Day 20 code breaks a rule Day 4 enables B16	Add <code>strict=True</code> . Do not suppress the rule.
<code>dvc repro</code> fails with <code>ModuleNotFoundError</code>	DVC spawned a different <code>python</code>	<code>dvc.yaml</code> hardcodes the name. Run from the activated environment.
Only four classes in <code>dvc</code> <code>metrics show</code>	Transcription dropped a template block D1	Restore <code>shipping</code> ; move technical templates out of <code>account</code> .
<code>metrics.json</code> never appears in a commit	<code>metrics/*.json</code> is ignored A6	Remove that line; <code>cache: false</code> needs Git to own it.
<code>git add data models</code> exits 1	Those directories are ignored B1	Ignore contents, not directories, with <code>.gitkeep</code> negations.
<code>FileNotFoundError:</code> <code>'params.yaml'</code>	Config path relative to the working directory B5	Anchor with <code>Path(__file__).resolve().parents[2]</code> .
Field "model_path" ... protected namespace	Pydantic v2 reserves <code>model_</code> B6	<code>ConfigDict(protected_namespaces=())</code> on each model.
Object of type float64 is not JSON serializable	A numpy scalar reached <code>json.dump</code> B7	Cast with <code>float(...)</code> where the metric is computed.
<code>ModuleNotFoundError: No</code> <code>module named</code> <code>'ticket_triage'</code>	You followed the other PDF for a step C4	Archive it. The package is <code>automated_support_ticket_classification</code> .
<code>pip install -e .</code> succeeds but the import fails	No <code>__init__.py</code> , so zero packages found B2	Create it, then reinstall.
Smoke test fails every morning	Single request cannot survive a cold start B17	Retry with a bounded loop. If you are not deployed, drop the schedule entirely.
Badge image is broken on the README	Repo name case does not match B15	Use the real repo name. Check the URL returns 200 before committing.

Container is <code>Up</code> but <code>curl</code> returns nothing	uvicorn has not bound the port yet	Poll for a few seconds rather than issuing one request.
<code>docker info</code> hangs, engine never starts	The <code>docker-desktop</code> WSL distro is stopped	Kill <code>Docker Desktop</code> and <code>com.docker.backend</code> , run <code>ws1 --shutdown</code> , relaunch. Back in ~30s.
Ugly traceback when you press Ctrl+C on uvicorn	<code>--reload</code> shutdown on Windows	Cosmetic. Drop <code>--reload</code> unless you are editing source.
Browsing <code>/</code> shows only a small JSON blob	That is the root endpoint, by design	The interactive UI is at <code>/docs</code> . The <code>/favicon.ico</code> 404 is normal.
<code>Activate.ps1</code> is blocked	PowerShell execution policy	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> .
pytest cannot import from <code>src</code>	<code>src</code> not on the path	The <code>pythonpath = ["src"]</code> block fixes it.
Docker: connection refused	uvicorn bound to 127.0.0.1	Bind <code>0.0.0.0</code> and map <code>-p 8000:8000</code> .
CI fails on the format check	Code is not ruff-formatted	<code>ruff format .</code> locally and commit.

Part 6 · Verification matrix

Day	Run	Expected
Repair	<code>pytest</code>	<code>13 passed</code> at this point, no <code>OptimizeWarning</code>
Repair	<code>ruff check . ; ruff format --check .</code>	All checks passed, 0 to reformat
Repair	<code>dvc --version</code>	<code>3.55.2</code> , no traceback
Repair	<code>git ls-files data models</code>	only <code>.gitkeep</code> entries
14	<code>python -m ...models.train</code>	a second run in <code>mlflow ui</code>
15	<code>dvc remote list</code>	<code>localremote ../dvc-storage</code>
16	<code>dvc dag</code>	generate, preprocess, train, evaluate
17	<code>dvc repro</code>	four stages run, <code>dvc.lock</code> written
17	<code>dvc metrics show</code>	five classes, supports summing to 600
18	<code>dvc status -c</code>	cache and remote in sync
19	<code>TicketRequest(text='hi')</code>	<code>text='hi'</code>
20	POST <code>/predict</code>	label, confidence, five scores summing to 1
21	GET <code>/</code>	service, docs and health keys
22	<code>pytest</code>	<code>13 passed</code>
22	<code>pytest</code> with no <code>model.joblib</code>	still 13 passed, fixture builds one
23	<code>docker build</code>	exports image, context a few kB
24	<code>docker run</code> , then <code>/health</code>	<code>ok</code> , after a few seconds
24	<code>docker run -e PORT=9001</code>	uvicorn binds 0.0.0.0:9001
25	<code>git push</code>	<code>lint-and-test</code> green first run
26	<code>git push</code>	both jobs green, docker after tests
27	<code>/metrics</code> after traffic	<code>http_requests_total</code> counters
27	<code>:9090/api/v1/targets</code>	target <code>health: UP</code> , no error
28	optional; skipped by design	blueprint committed and valid
29	run Smoke Test manually	guard fires cleanly with no secret set
30	<code>git push origin v1.0.0</code>	tag visible, badge green, release published
8.1	<code>dvc repro</code> with <code>source: banking77</code>	13,083 rows, accuracy 0.9282, macro F1 0.9171
8.2	<code>python -m ...models.promote</code>	promotes, or exits 2 with a reason
8.3	<code>python -m ...monitoring.drift</code>	<code>status: ok</code> above 200 rows, <code>insufficient_data</code> below
now	<code>pytest</code>	<code>30 passed</code>

Final checklist

- ☑ Repo initialised with README, LICENSE, .gitignore
- ☑ Virtual environment, pinned dependencies *including transitive pins*, editable install
- ☑ No declared dependency the project does not import
- ☑ ruff clean: `check` and `format --check` both pass
- ☑ params.yaml plus typed config, paths anchored to the project root
- ☑ Reproducible data generation covering all five classes
- ☑ Training with an sklearn Pipeline and MLflow tracking
- ☑ Evaluation writing a Git-tracked metrics.json
- ☑ 13 tests passing, including the whitespace and label-set guards
- ☑ DVC initialised, pipeline defined, dvc.lock committed, artifacts pushed
- ☑ Data artifacts owned by DVC and nothing else
- ☑ FastAPI service sharing clean_text with training
- ☑ Service verified by running it, one ticket per class
- ☑ Multi-stage Dockerfile building from requirements-serve.txt
- ☑ CI green on every run: format, lint, test, build, container smoke test
- ☑ Prometheus instrumentation exposed at /metrics
- ☑ Polished README stating the local-only scope, with a working badge
- ☑ v1.0.0 tagged, pushed and released
- ☑ Trained on a real corpus, with the n-gram question re-answered on it
- ☑ Model registry with corpus-aware, metric-gated promotion
- ☑ Drift monitoring with a minimum-sample gate
- ☑ Prometheus scrape verified end to end, target UP and counters stored
- ☑ **N/A** Deployed to a public URL (*deliberately out of scope*)

Part 7 · Interview preparation

Core questions

Q: What is MLOps in one sentence?

A: Applying software engineering and DevOps discipline, meaning versioning, testing, CI/CD, deployment and monitoring, to the full machine learning lifecycle so models ship reliably and stay healthy in production.

Q: Walk me through the architecture.

A: A DVC pipeline generates synthetic ticket data, preprocesses and splits it, trains a TF-IDF plus Logistic Regression pipeline tracked in MLflow, and evaluates it into a versioned metrics file. The saved sklearn Pipeline is served by FastAPI, containerised, linted, tested and built by GitHub Actions, and instrumented for Prometheus.

Q: How is it reproducible?

A: Code by Git, data by a seed and DVC, hyperparameters by params.yaml, the environment by pinned requirements, and the exact run by hashes in dvc.lock. I would add that pinning direct dependencies is not the same as pinning the graph, which I learned the hard way on this project.

Q: How do you prevent data leakage?

A: Split before fitting anything, and keep every transform inside a Pipeline fit only on the training fold. The Pipeline also guarantees the identical transform at inference, which is the other half of the same problem.

Q: What do you monitor?

A: Operationally the RED signals: request rate, error rate, duration. For ML specifically you would add input and prediction drift and data-quality checks, which is a separate concern from service health and needs separate tooling.

Q: Why is it not deployed?

A: A deliberate scope decision. It is a portfolio build, not a product with users, so paying for and babysitting a hosted instance buys nothing. The deployment path is built and committed, a Render blueprint and a post-deploy smoke workflow, so deploying is a configuration step rather than engineering work. I would rather ship something honest that runs in two commands than a dead demo URL on a sleeping free tier.

Q: What are the weaknesses of this design?

A: Synthetic data, no model registry, no drift detection, and a single-node serving model. Each is a deliberate scope decision with a named next step.

The questions this build actually earned

Q: Tell me about a data quality problem you found.

A: My classifier was supposed to have five categories. Reading per-class metrics after wiring up the pipeline, I noticed only four, with supports summing to exactly my test size. A page break in the source material had dropped one category's templates entirely and filed another category's under the wrong label, so a quarter of my training data was contradicting itself. Restoring it took accuracy from 0.9217 to 0.9300 and lifted the affected class's recall from 0.869 to 0.920. What is instructive is that nothing caught it: the training logs said 0.92, and my tests asserted shapes and column names, not the label set. Only per-class metrics made it visible. I added a test pinning the label set so it cannot come back.

Q: Describe a time the obvious fix was wrong.

A: Every model fit emitted a warning from a scikit-learn file about an unknown solver option. The instinct is to upgrade scikit-learn. I checked the source of every release from 1.5 through 1.9 and all of them still send that option; it had been removed on the SciPy side between 1.17 and 1.18. Pinning SciPy fixed it and upgrading scikit-learn would have changed nothing. Find the layer that actually changed before you change anything.

Q: Have two tools ever fought over the same resource?

A: Yes. My data files were committed to Git before I introduced DVC, and `dvc repro` refused to run: an output

cannot be tracked by SCM and by DVC at once. That is correct behaviour, because two version control systems cannot both own one file's history. The subtlety in the fix is that Git will not descend into an ignored directory, so you have to ignore the contents and negate the placeholder rather than ignoring the directory.

Q: What is train/serve skew and did you hit it?

A: It is when inference applies a different transformation from training. I had a mild case: training normalised with a function that lowercased and collapsed whitespace, while the API endpoint did its own strip and lower. The practical impact was near zero with TF-IDF, but two independent definitions of "clean" guarantee divergence as preprocessing grows, so I made the API import the same function. The structural fix is what the Pipeline already does for the model: one artifact, one transform, no second implementation to drift.

Q: How do you keep a production image small?

A: Multi-stage build, slim base, and a separate runtime requirements file. My training code tracks experiments in MLflow, but MLflow has no role at serving time, so the image installs a serve-only requirements file that omits it. Worth noting: I originally had an env flag disabling MLflow, which stopped it being used but not installed. The flag was treating the symptom.

Q: How do you decide what belongs in a requirements file?

A: I audited declared dependencies against actual imports and found one the project never used, a leftover from the tutorial I started from. Removing it changed nothing functionally, which is exactly the point: a requirements file is a claim about what the code needs, and a claim nobody checks is one nobody can trust. The same audit is what surfaced the transitive pins I was missing.

Closing note. The defects in this document are not a sign the original guide was bad. Every long technical document accumulates them, and finding them is the job. What matters is that you hit a failure, read the traceback rather than the tutorial, and fixed the cause instead of the symptom.

Part 8 · Beyond Day 30

The 30-day guide ends with a working, tested, containerised service and lists further work as "future improvements". Three of those were then built. This part covers what they are, what they cost, and what each one taught, because in every case the interesting output was not the feature but the defect it exposed.

State after this part. 49 commits. 30 tests. Trained on 13,083 real support messages rather than templates: accuracy 0.9282, macro F1 0.9171. Six model versions registered with gated promotion. Drift monitoring over live traffic. CI green throughout.

8.0 · A test page and a one-command launcher

Smaller than the other three, and the one that changes daily use most. FastAPI's `/docs` works but takes four clicks per request and prints the probability distribution as raw JSON. `/ui` is a hand-written page with one preset ticket per category, a free-text box, and the distribution drawn as bars so the runner-up classes are visible at a glance.

The HTML lives in a Python module rather than a static file, so it needs no package-data declaration and no extra `COPY` in the Dockerfile. No CDN and no build step: it works offline and inside the container. It is excluded from the OpenAPI schema, being a convenience for humans rather than part of the API contract.

`run.ps1` finds the interpreter without the environment being activated, refuses to start if the port is already listening, waits until `/health` answers, then opens the page. `run.cmd` wraps it for double-clicking.

A bug the tests could not have caught. The first render drew every bar at full width regardless of score. `.track` and `.fill` are `span` elements, and width and height do not apply to inline elements. The tests passed throughout: they asserted the page returned 200 and contained the category names, both true of a visually broken page. For anything visual, rendering it is the test.

8.1 · Training on a real corpus

THE CANDIDATE THAT FAILED. The CFPB consumer-complaints database is the obvious public dataset for this task. It is unusable: its 347 MB bulk export contains no complaint narrative, and its search API rejects `complaint_what_happened` as an invalid field. Both were checked directly. The dataset has categories but no text to classify.

WHAT REPLACED IT. banking77: 13,083 real customer-support messages labelled with 77 fine-grained intents, 1 MB. Its intents fold onto the five categories, and critically they include genuine card-delivery intents, which is what makes a `shipping` class possible at all.

The mapping lives in `data/banking77.py` with every borderline call marked. The rule applied throughout is *what would a support team need to do about this*, which is what a routing model is for. It is a judgement and a reviewer is entitled to disagree with any of it.

CLASS BALANCE, DELIBERATELY NOT RESAMPLED. billing 37.8%, account 22.6%, technical 16.6%, general 15.4%, shipping 7.6%. Real support traffic is skewed, and flattening it would hide exactly the effect macro F1 exists to measure.

THE SOURCE IS A TRACKED PARAM. `data.source` selects `synthetic` or `banking77` and is declared in `dvc.yaml`, so switching corpus reruns the pipeline. Synthetic stays the default because CI and the Docker build have no network.

The result that reversed Day 14

ngram_max	accuracy	macro F1	verdict
1	0.9201	0.9100	baseline

2	0.9282	0.9171	adopted
3	0.9251	0.9139	worse than 2

On templates, unigrams and bigrams scored *identically*, to four decimal places. On real text bigrams win by 0.71 points of macro F1, and trigrams hand the gain back, which is the usual shape of an n-gram sweep: more context helps until the features get too sparse to estimate.

The lesson is about experiments, not n-grams. Day 14's conclusion was never about bigrams. It was about the dataset. An experiment run on data that cannot distinguish two options will always report that the two options are the same, and it will look like a finding. The tie should have been read as a warning about the data, not as an answer about the model.

Note the scores are *lower* than the synthetic 0.9300 / 0.9297, and that is correct. The synthetic figure measured how well a model can learn five template sets, which was never a hard question.

8.2 · A model registry with gated promotion

Without a registry, "the model in production" means "whatever the last `dvc repro` happened to produce", which is not a decision anyone made. Training now registers every run as a version tagged with its metrics. Promotion is a separate, deliberate act.

```
python -m automated_support_ticket_classification.models.promote
```

Design choices worth defending:

- **Aliases, not the deprecated Staging/Production stages.** An alias is a movable pointer to a version, so promotion and rollback are the same one-line operation in opposite directions.
- **Gated on macro F1, not accuracy.** Accuracy can rise while a minority class collapses, which is exactly how the four-class data defect hid for so long.
- **A tie does not promote.** The incumbent is already known to work; an equal model costs a deploy and gains nothing.
- **Exits 2 when nothing ships,** so CI can branch on it without parsing logs. That is not a failure: "the incumbent is still better" is a result.

The defect it exposed, one commit later

The gate was comparing incomparable numbers. The first real-data model scored macro F1 0.9171. Production held the synthetic-data model at 0.9297. The gate compared the two and refused to promote.

That comparison is meaningless. The scores come from different test sets: one measures 2,617 real support messages, the other 600 generated from templates. They are not on the same scale.

THE PART THAT MAKES IT WORSE THAN A WRONG ANSWER. It refused, which looks like caution. Had the synthetic number happened to be lower, the gate would have promoted the worse model just as confidently, on exactly the same meaningless comparison. It was not being careful; it had no idea what it was comparing. **A gate that gives the right answer for no reason will give the wrong answer as soon as the numbers move.**

FIX. Every version carries a tag for the corpus that produced its score, and the gate only compares like with like. When the corpus changes, the incumbent is not evidence of anything, so the candidate takes over and normal comparisons resume from there. Two tests pin both directions, because the corpus escape hatch must not become a way to ship anything at all.

Generalisation. A metric gate is only as meaningful as the comparability of the two numbers it compares, and a bare float carries no record of how it was measured. Tag the score with what produced it, or the gate is theatre.

8.3 · Drift monitoring

Prometheus answers whether the *service* is healthy: rate, errors, duration. It cannot tell you the service is cheerfully returning 200s while the traffic has changed underneath it.

Two signals are tracked separately, because they fail differently:

Signal	What it means when it moves
label drift	The model changed its mind about the mix it is emitting.
text drift	The input changed first. Earlier, and the more useful warning.

A single dataset-level boolean hides which one moved, so each column carries its own verdict, threshold and method. The API appends every prediction to `data/predictions.jsonl`, which is what gives drift a real population to measure; the write is best effort inside a `try/except`, because a monitoring side effect must never be able to fail a request.

VALIDATED IN BOTH DIRECTIONS, which is the part that matters for a detector, since one that never fires is worthless:

```
train vs test, same corpus    label 0.0003, text 0.5089 -> quiet
real vs synthetic templates   label 0.1758, text 0.9969 -> both fire
```

The defect this one exposed

It cried wolf on tiny samples. With five predictions logged it reported label drift of 0.1764 against a 0.10 threshold. With seventeen, 0.2662. Both looked like a real shift.

Neither was. Five predictions spread one per category look dramatically drifted against a training set that is 38% billing and 8% shipping, purely because five is far too few to estimate a share from. The monitor was detecting that it had barely looked at the world, not that the world had changed.

Left alone this is worse than no monitor: it would fire after every deploy, and the first thing anyone does with an alarm that is usually wrong is stop reading it.

FIX. `MIN_SAMPLE_ROWS = 200`, chosen so the natural class balance still leaves roughly 40 rows in the smallest class. Below it the scores are still computed and reported, because they are the evidence, but no verdict is issued.

rows	status	dataset_drifted
17	<code>insufficient_data</code>	<code>null</code>
300	<code>ok</code>	<code>false</code>

Null rather than false, and a test pins the difference. A monitor keying on `dataset_drifted` alone would read an unknown as a clean bill of health, which is how a monitor goes silent without anyone noticing. The rule is: alert only when `status` is `ok` and `dataset_drifted` is true.

Generalisation. Every statistical alarm needs a floor on the evidence it may fire from. A threshold answers "how big a difference matters"; it says nothing about "how much data before the question is worth asking". Both are needed, and only one of them tends to get written down.

What this part is worth in an interview

Each of the three features shipped with a defect, and each defect is a better answer than the feature:

- **"Tell me about a time your own code was subtly wrong."** The promotion gate refused a model for a reason that did not hold, and looked careful while doing it.
- **"How do you know your monitoring works?"** By making it fire on purpose, and by checking it stays quiet when it should. A detector validated in only one direction is not validated.
- **"When is a result too good to trust?"** When two options score identically to four decimal places, suspect the data before believing the finding.